Server-side route protection

How login, the cookie, and the redirects are wired in this project. Every code block below is the code currently in the repo.

Account: `sreekanth@gmail.com` / `123123123`. Cookie name: `ecom-auth`. Cookie value: `123123123`. That value must be the same in the login action, `session.ts`, and `proxy.ts`.

Files

1. `src/features/auth/action/login.action.tsx` sets the cookie
2. `src/features/auth/components/LoginForm.tsx` calls that action
3. `src/features/auth/session.ts` reads the cookie
4. `src/app/products/layout.tsx` blocks `/products` and `/products/[id]`
5. `src/app/(auth)/layout.tsx` sends a logged-in user away from `/login`
6. `src/proxy.ts` redirects before the page runs
7. `src/features/auth/action/logout.action.ts` deletes the cookie
8. `src/features/products/components/ProductsHeader.tsx` calls logout

1. What this setup does

The old guard lived in the browser. `RouteGuard` read `localStorage` inside `useEffect` and then called `router.replace`. The server had already sent the page. The new setup stores login in an `httpOnly` cookie and checks that cookie on the server before the protected page is sent.

```
Browser submits the form
        |
        v
loginAction runs on the server
        |  password matches
        |  response header: Set-Cookie ecom-auth=123123123; HttpOnly
        v
Browser stores the cookie and goes to /products
        |
        v
src/proxy.ts reads the cookie
        |  missing or wrong value -> /login
        v
products/layout.tsx calls requireUser()
        |  missing or wrong value -> /login
        v
{children} renders the products page
```

`{children}` is not assigned in the login form. `router.replace("/products")` changes the URL. Next.js then puts `src/app/products/page.tsx` into the layout's `children`. A click to `/products/1` puts `src/app/products/[id]/page.tsx` into the same slot. Both URLs use `src/app/products/layout.tsx`, so both are protected by one `requireUser()` call.

URL	Protected?	Where
`/login`	Logged-in users are sent to `/products`	`proxy.ts` and `(auth)/layout.tsx`
`/products`	Logged-out users are sent to `/login`	`proxy.ts` and `products/layout.tsx`
`/products/1`	Same as `/products`	Same products layout
`/` and `/cart`	Open	No session check

> The cookie value `123123123` is written in three files. If login stores one string and `session.ts` or `proxy.ts` expects another, the cookie appears in DevTools and the browser stays on `/login`. `proxy.ts` treats the user as logged out and sends `/products` back to `/login`.

2. Set the cookie

src/features/auth/action/login.action.tsx

```
"use server";

import { cookies } from "next/headers";
import { loginSchema } from "../schemas/login.schema";

type LoginActionState = { success: true } | { success: false; error: string };

export default async function loginAction(
  values: unknown,
): Promise<LoginActionState> {
  const parsed = loginSchema.safeParse(values);

  if (!parsed.success) {
    return {
      success: false,
      error: "Please enter valid email and password.",
    };
  }

  const { email, password } = parsed.data;

  const isValidUser =
    email === "sreekanth@gmail.com" && password === "123123123";

  if (!isValidUser) {
    return { success: false, error: "Invalid email or password." };
  }

  const cookiestore = await cookies();

  cookiestore.set({
    name: "ecom-auth",
    value: "123123123",
    httpOnly: true,
    sameSite: "lax",
    path: "/",
    maxAge: 60 * 60 * 24,
  });

  return { success: true };
}
```

`"use server"` keeps this function on the server. `cookies()` is async in this Next.js version. `.set()` is allowed here because a Server Action can write response headers. A Server Component can read a cookie and cannot set one.

- `httpOnly: true` hides the cookie from `document.cookie` and from page JavaScript.
- `sameSite: "lax"` sends the cookie on normal visits to this site.
- `path: "/"` sends it for every route.
- `maxAge: 60 * 60 * 24` keeps it for one day.
- `secure` is omitted so the cookie is stored on `http://localhost`.

The submit handler in src/features/auth/components/LoginForm.tsx

```
const onSubmit = async (values: LoginFormValues) => {
  const result = await loginAction(values);
  if (!result.success) {
    setServerError(result.error);
    return;
  }
  router.replace("/products");
};
```

The form calls `loginAction(values)`. It is not placed on `<form action={...}>`, because that would pass `FormData` and the Zod parse in the action would fail. After `{ success: true }`, `router.replace("/products")` changes the URL. The cookie is already on the response from the action.

3. Read the cookie

`server-only` is installed with `pnpm add server-only`. Importing it from a Client Component fails the build. This project's `pnpm` on the default PATH is version 9, which rejects `pnpm-workspace.yaml`. The install that succeeded used Node 24 and pnpm 11.22.0:

```
export PATH="$HOME/.nvm/versions/node/v24.22.1/bin:$PATH"
corepack pnpm add server-only
```

src/features/auth/session.ts

```
import "server-only";
import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function getSession() {
  const token = (await cookies()).get("ecom-auth")?.value;
  if (token !== "123123123") return null;
  return { userId: "student" };
}

export async function requireUser() {
  const session = await getSession();
  if (!session) redirect("/login");
  return session;
}
```

`getSession()` returns null when the cookie is missing or the value is not `123123123`. `requireUser()` turns that null into a redirect. `redirect()` throws, so the layout stops and the products UI is not sent.

4. Protect the products pages

src/app/products/layout.tsx — the active code. The old RouteGuard wrapper is commented out above it.

```
import { requireUser } from "@/features/auth/session";
import ProductsHeader from "@/features/products/components/ProductsHeader";

export const instant = false;

export default async function ProductsLayout({
  children,
}: LayoutProps<"/">) {
  await requireUser();

  return (
    <div className="flex grow flex-col bg-neutral-50 text-neutral-900">
      <ProductsHeader />
      <main className="grow">
        <div className="mx-auto max-w-6xl px-4 py-8">{children}</div>
      </main>
    </div>
  );
}
```

`await requireUser()` runs before `{children}`. A logged-out request never receives the products page. `export const instant = false` is required because `next.config.ts` has `cacheComponents: true`. Reading `cookies()` cannot be prerendered into the static shell, so this layout opts out and waits for the request.

`/products` fills `children` with `src/app/products/page.tsx`. `/products/1` fills it with `src/app/products/[id]/page.tsx`. The header, including Logout, is outside `children` and is still behind the same `requireUser()` call.

5. Keep a logged-in user off /login

src/app/(auth)/layout.tsx

```
import { getSession } from "@/features/auth/session";
import { redirect } from "next/navigation";

export const instant = false;

export default async function AuthLayout({ children }: LayoutProps<"/">) {
  const session = await getSession();
  if (session) redirect("/products");

  return (
    <main className="grow flex flex-col">
      <div className="grid grid-cols-2 grow">
        <section className="bg-red-500 h-full grid place-items-center">
          <div>
            <h1 className="text-center text-5xl">Move fast when you</h1>
            <h1 className="text-center text-5xl">break things</h1>
          </div>
        </section>
        <section className="h-full grid place-items-center bg-white text-neutral-900">
          {children}
        </section>
      </div>
    </main>
  );
}
```

`(auth)` is a route group. It does not appear in the URL. This layout wraps `src/app/(auth)/login/page.tsx`, which is `/login`. That page sets `export const instant = false` for the same Cache Components reason and renders `<LoginForm />`.

When the cookie matches, `getSession()` returns `{ userId: "student" }` and `redirect("/products")` runs. The form is not rendered.

6. Redirect early in proxy.ts

Next.js 16 uses `src/proxy.ts`. Older docs call this file `middleware.ts`. It runs before the page, reads the request cookie, and redirects. It does not create the `/login` route. That URL still comes from the `login` folder.

src/proxy.ts

```
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";

export function proxy(request: NextRequest) {
  const isLoggedIn = request.cookies.get("ecom-auth")?.value === "123123123";
  const { pathname } = request.nextUrl;

  if (pathname.startsWith("/products") && !isLoggedIn) {
    return NextResponse.redirect(new URL("/login", request.url));
  }

  if (pathname === "/login" && isLoggedIn) {
    return NextResponse.redirect(new URL("/products", request.url));
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/products/:path*", "/login"],
};
```

`requireUser()` in the products layout is the check that matters. The proxy is an earlier redirect so a logged-out request to `/products` does not start rendering that page. Restart `pnpm dev` after adding this file so Next.js loads it.

Both checks must use `123123123`. A cookie that exists but has another value makes `isLoggedIn` false, and `/products` is sent back to `/login`.

7. Log out

src/features/auth/action/logout.action.ts

```
"use server";

import { cookies } from "next/headers";
import { redirect } from "next/navigation";

export async function logoutAction() {
  (await cookies()).delete("ecom-auth");
  redirect("/login");
}
```

Deleting the cookie is allowed in a Server Action. The next request has no `ecom-auth` value, so `getSession()` returns null and `proxy.ts` treats the user as logged out.

The button in src/features/products/components/ProductsHeader.tsx

```
<Button
  onClick={async () => await logoutAction()}
  className="rounded-md px-3 py-1.5 text-sm text-white"
>
  Logout
</Button>
```

`ProductsHeader` is a Client Component (`"use client"`). It can call `logoutAction` . The button's `className` is merged by the shared `Button` component. `text-white` is required because the default variant only sets the dark background.

8. Request paths

Logged out, open /products

1. The browser requests `/products` with no `ecom-auth` cookie.
2. `proxy.ts` sees `pathname.startsWith("/products")` and `isLoggedIn === false`.
3. It returns a redirect to `/login`.
4. The login layout calls `getSession()`, gets null, and renders the form.

Submit the correct password

1. `LoginForm` calls `loginAction` with the email and password.
2. The action checks `sreekanth@gmail.com` / `123123123`.
3. The action sets `ecom-auth=123123123` and returns `{ success: true }`.
4. The form calls `router.replace("/products")`.
5. `proxy.ts` sees the matching cookie and calls `NextResponse.next()`.
6. `requireUser()` sees the same value and returns `{ userId: "student" }`.
7. The layout renders `{children}`, which is the products page.

Already logged in, open /login

1. `proxy.ts` matches `pathname === "/login"` and `isLoggedIn`.
2. It redirects to `/products`.
3. If that proxy check were skipped, the login layout's `getSession()` would redirect as well.

Logout

1. The header calls `logoutAction()`.
2. The action deletes `ecom-auth` and redirects to `/login`.
3. A later visit to `/products` is redirected to `/login` again.

9. How to test

Use a private window so an old cookie is not already stored. Restart the dev server after `src/proxy.ts` is added.

1. Open `/products`. The address becomes `/login`. The checking sentence from `RouteGuard` does not appear, because that component is no longer rendered.
2. Open `/products/1`. The address becomes `/login`.
3. Submit a wrong password. The form stays on `/login` and shows “Invalid email or password.”
4. Sign in with `sreekanth@gmail.com` / `123123123`. The address becomes `/products`.
5. In DevTools → Application → Cookies, `ecom-auth` is `123123123` and HttpOnly is checked.
6. In the browser console, `document.cookie` does not list `ecom-auth`.
7. Refresh `/products`. The products page stays.
8. Open `/login`. The address returns to `/products`.
9. Click Logout. The address is `/login` and the cookie is gone.

`/` and `/cart` still open while logged out. They do not call `requireUser()` and they are not in the proxy `matcher`.